

Architettura degli elaboratori e sistemi operativi

Diego Stefanini — prof. Marco Danelutto, a.a. 2026-27

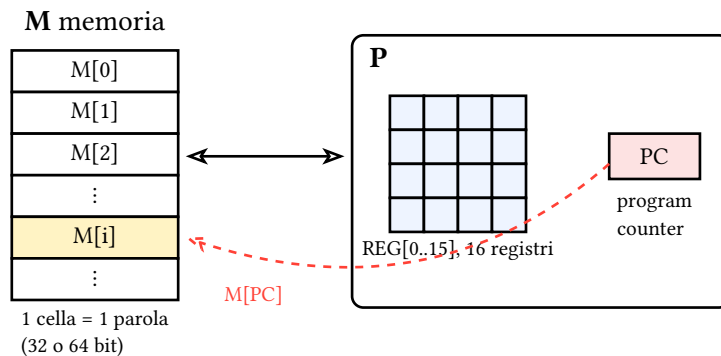
Indice

1 Fondamenti dei sistemi di elaborazione	3
1.1 Memoria, processore, istruzioni	3
1.2 Astrazione	4
1.3 Numeri binari	5

1 Fondamenti dei sistemi di elaborazione

1.1 Memoria, processore, istruzioni

Un calcolatore sono due pezzi: la **memoria** M e il **processore** P, che si parlano.



Il PC contiene l'indirizzo della prossima istruzione da prendere in memoria

Una volta avviato, il processore ripete per sempre questi quattro passi:

```
while (true) {  
    preleva l'istruzione da M[PC]  
    decodifica      // che istruzione è?  
    esegue  
    scrive i risultati  
}
```

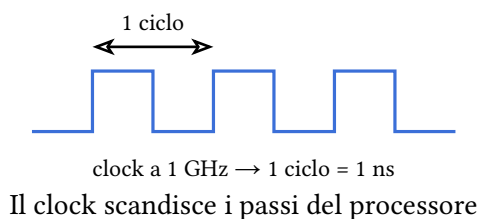
Quali istruzioni esistono dipende dal processore (x86, ARM, RISC-V), ma i tipi sono sempre tre:

Tipo	Istruzioni	Esempio
operative	ADD SUB MUL DIV AND OR NOT SHIFT ROT (sx/dx)	ADD R0, R1, R2 → REG[0] = REG[1] + REG[2]
memoria	LOAD STORE	LOAD R5, addr → REG[5] = M[addr] STORE fa il contrario: registro → memoria
salto	salto	cambia il PC: la prossima istruzione non è quella dopo

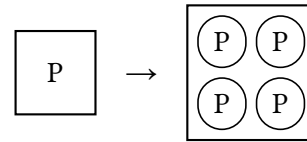
```
inizio: ADD R0, R0, R0 ← PC  
        ADD R1, R1, R1  
        SUB R2, R0, R1  
        salto inizio  
        ↑
```

Il PC scende di un'istruzione alla volta; il **salto** lo rimette su `inizio`, quindi il programma ricomincia.

A dare il ritmo ai passi del ciclo è il **clock**:



- 1 GHz = 10^9 cicli al secondo, quindi un ciclo dura 10^{-9} s = 1 ns.
- Transistor: nel 68000 20-40 μm , oggi 2 nm ($1000 \text{ nm} = 1 \mu\text{m}$).
- Transistor più piccoli → nello spazio di un processore ne stanno più di uno (**multico-re**):



Da sapere — legge di Moore (aggiunto, non spiegato in aula)

Il numero di transistor che si riescono a mettere su un chip raddoppia circa ogni due anni. È per questo che si passa dai μm ai nm e da un processore a più processori sullo stesso chip.

1.2 Astrazione

Un calcolatore si studia a **livelli di astrazione**: ognuno usa quello sotto senza doverne conoscere i dettagli.

Applicazioni
Sistema operativo — processi, memoria, file system
Architettura — set di istruzioni (ARMv7)
Microarchitettura — il processore
Logica — componenti: AND, OR, NOT, «memoria»
Circuiti digitali — segnali «1» e «0»: FA, MUX
Circuiti analogici
Dispositivi
Fisica — questi ultimi 3 non li facciamo

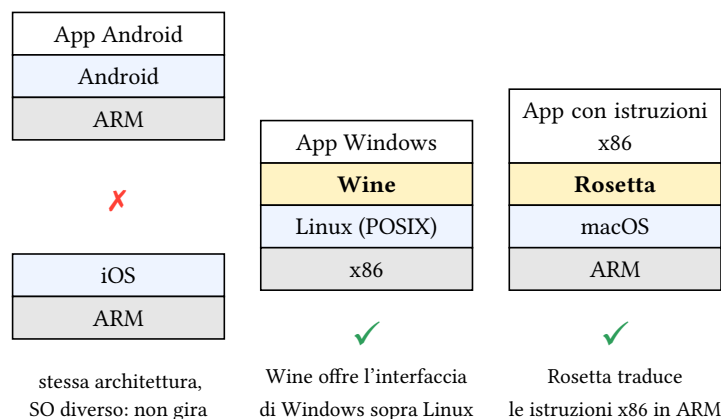
Tra un livello e l'altro c'è un'**interfaccia**:

Applicazione — livello i+1
interfaccia: chiamate al sistema operativo
Sistema operativo — livello i
interfaccia: istruzioni in linguaggio macchina
Architettura — livello i-1

Fra due livelli c'è un'**interfaccia**: l'insieme delle funzionalità che il livello i-1 mette a disposizione del livello i.

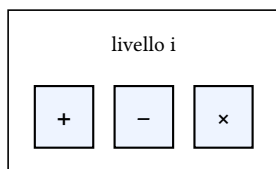
Il livello i usa solo l'interfaccia, non sa com'è fatto sotto.

Conseguenza: un programma gira solo dove trova l'interfaccia per cui è stato scritto.

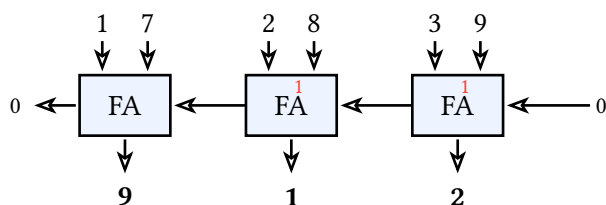


Per costruire sistemi così complessi si usano tre principi:

- **Gerarchia**: il sistema è diviso in livelli (sopra).
- **Modularità**: ogni livello è fatto di moduli con un compito preciso.
- **Regolarità**: lo stesso modulo si riusa tante volte.



Modularità: un livello fatto di moduli

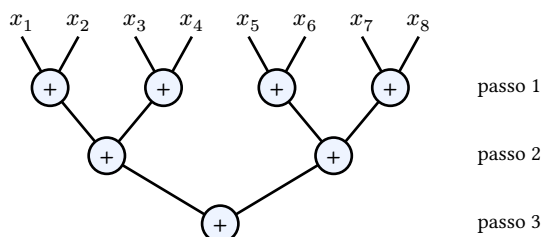


Regolarità: $123 + 789 = 912$ con tre FA uguali in cascata.
Ogni FA somma una cifra e passa il riporto a sinistra

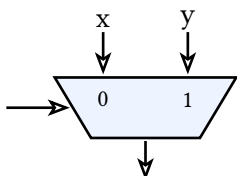
Stesso modulo $+$, due modi di sommare 8 numeri:

```
sum = 0;
for (i = 0; i < n; i++)
    sum = sum + x[i];
```

7 somme **una dopo l'altra**: ognuna aspetta la precedente.



Ad albero: le somme dello stesso livello sono indipendenti, quindi le stesse 7 somme stanno in 3 passi



MUX: il segnale che entra di lato sceglie quale ingresso passa in uscita, x (se vale 0) o y (se vale 1).

1.3 Numeri binari

I circuiti digitali lavorano in **binario**: ogni segnale vale 0 o 1.

In un sistema posizionale **conta l'ordine delle cifre**: la stessa cifra vale di più quanto più è a sinistra. Ogni posizione ha un peso.

decimale	$10^3 = 1000$	$10^2 = 100$	$10^1 = 10$	$10^0 = 1$
binario	$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$

Esempio: $13_{10} \neq 31_{10}$ e $10_2 \neq 01_2$. Stesse cifre, ordine diverso, numero diverso.

Per passare **da decimale a binario** divido per 2 finché arrivo a 1, poi leggo i resti **dal basso verso l'alto**.

n	: 2	resto
12	6	0
6	3	0
3	1	1
1		1 ↑

$$12 = 1100_2$$

n	: 2	resto
7	3	1
3	1	1
1		1 ↑

$$7 = 111_2$$

Somma e prodotto in binario si fanno come in decimale, ma $1 + 1 = 10_2$: scrivo 0 e riporto 1.

$$\begin{array}{r}
 8421 \\
 1010 \\
 + 0100 \\
 \hline
 1110
 \end{array}
 \qquad
 \begin{array}{r}
 111 \\
 0111 \\
 + 0001 \\
 \hline
 1000
 \end{array}$$

$10 + 4 = 14$
 $7 + 1 = 8$ (riporti in grigio)

Il prodotto si fa in colonna come in decimale. In binario ogni riga è molto semplice: la cifra del moltiplicatore è 0 o 1, quindi la riga è tutta zeri oppure il numero stesso. È un **AND** bit a bit.

$$\begin{array}{r}
 12 \\
 \times 34 \\
 \hline
 48 \\
 36 \cdot \\
 \hline
 408
 \end{array}
 \qquad
 \begin{array}{r}
 1010 \\
 \times 0100 \\
 \hline
 0000 \\
 0000 \cdot \\
 1010 \cdot \cdot \\
 0000 \cdot \cdot \cdot \\
 \hline
 0101000
 \end{array}$$

in decimale
 $10 \times 4 = 40 = 32 + 8$

Per i **numeri negativi** il modo più semplice è **modulo e segno**: il primo bit è il **segno** (0 = +, 1 = -), gli altri sono il **valore assoluto**.

1	1	1	0	0
---	---	---	---	---

segno valore assoluto = 12

-12 in modulo e segno

Per sommare A e B bisogna prima guardare i segni:

Caso	Risultato
$S_A = S_B$	segno S_A , valore $V_A + V_B$
$S_A \neq S_B$ e $V_A > V_B$	segno S_A , valore $V_A - V_B$
altrimenti	segno S_B , valore $V_B - V_A$

Esempi: $+12 + 12 = +24$, $-12 + (-12) = -24$, $-12 + 7 = -5$ (segni diversi: faccio $12 - 7$ e tengo il segno del più grande).

Servono confronti e sottrazioni: il circuito che somma diventa complicato. Il complemento a 2 risolve questo problema.

Si usa quindi il **complemento a 2**. Per passare da n a $-n$:

$$\boxed{n \text{ in binario}} \rightarrow \boxed{\text{nego tutti i bit}} \rightarrow \boxed{+1} \rightarrow \boxed{-n}$$

Vale anche al contrario: rifacendo «nego + 1» su $-n$ torno a n . Così, se un risultato comincia con 1 (è negativo), capisco quanto vale.

Il vantaggio: **la somma si fa come una somma normale**, senza guardare i segni.

Esempio: $-12 + 7$ su 5 bit.

12	01100		10100		somma 11011
	nego 10011		+ 00111		nego 00100
	+1 10100		11011		+1 00101
	-12		-12 + 7		inizia per 1, è negativo: vale -5 ✓

Esempio: $-5 + (-3)$ su 6 bit.

5	000101		3	000011		11111		111011		somma 111000
	nego 111010		nego 111100		+ 111101		111011		nego 000111	
	+1 111011		+1 111101		111000		111000		+1 001000	
	-5		-3		esce un riporto 1 oltre i 6 bit: si butta		vale 8, quindi il risultato è -8 ✓			